ZOIDBERG 2.0

Pneumonia Detection from Chest X-Ray Images

Full Project Flow & Team Division Document

Epitech Digital School — MSc Information Technology

Project Overview

Zoidberg 2.0 is a binary classification machine learning project. The goal is to analyze chest X-ray images and predict whether a patient has pneumonia or not. We use classical ML algorithms with proper data splitting, hyperparameter tuning, model evaluation, and model persistence.

Item	Detail
Project Name	Zoidberg 2.0 — Computer Aided Diagnosis
Goal	Binary classification: Normal vs Pneumonia
Input	Chest X-ray images (3 datasets)
Output	Predicted label + probability score
Main Metric	ROC-AUC Score
Team Size	2 people (Person A + Person B)
Deliverables	Jupyter Notebook + HTML export + PDF synthesis

Dataset Strategy

We have 3 datasets. Each one has a specific role in the project. Never mix them up.

Dataset	Source Folder	Role	When to Use
Dataset 1	train/	Training	Feed into model.fit()
Dataset 2	val/	Tuning / Validation	GridSearchCV, hyperparameter tuning

Dataset 3	test/	Final Evaluation	Only at the very end — never tune on this

Important: Dataset 3 (test) must never be touched during training or tuning. It is only used once at the very end to report final results.

Project Folder Structure

Folder / File	Purpose
data/dataset1/	Train images — NORMAL/ and PNEUMONIA/ subfolders
data/dataset2/	Val images — NORMAL/ and PNEUMONIA/ subfolders
data/dataset3/	Test images — NORMAL/ and PNEUMONIA/ subfolders
notebooks/zoidberg2_pneumonia.ipynb	Main Jupyter notebook — all code goes here
src/data_loader.py	Functions to load and read images
src/preprocess.py	Resize, normalize, flatten, PCA functions
src/train.py	Model training code
src/evaluate.py	Metrics, ROC curves, confusion matrix
src/utils.py	Helper functions
models/best_pipeline.joblib	Saved trained model pipeline
outputs/figures/	All saved charts and plots
report/summary.pdf	Final PDF synthesis document
requirements.txt	All Python dependencies

Team Division

Person A — Data & Preprocessing

Person A is responsible for everything related to raw data — loading, exploring, cleaning, and converting images into feature matrices ready for ML models.

Task 1 — Organize Datasets

- Copy train/ contents into data/dataset1/
- Copy val/ contents into data/dataset2/

- Copy test/ contents into data/dataset3/
- Verify each folder has NORMAL/ and PNEUMONIA/ subfolders

Task 2 — Exploratory Data Analysis (EDA)

- Count total images per dataset and per class
- Detect corrupt or unreadable images using try/except
- Check image sizes — are they all the same dimensions?
- Check class balance — is NORMAL vs PNEUMONIA roughly equal?
- Plot class distribution bar charts for all 3 datasets
- Show 5-10 sample X-ray images (normal vs pneumonia side by side)

Task 3 — Preprocessing Pipeline

- Convert all images to grayscale
- Resize all images to 128x128 pixels
- Normalize pixel values to range [0, 1] by dividing by 255.0
- Flatten each image from 128x128 grid to a 1D vector of 16,384 values
- Stack all vectors into matrix X of shape (n_samples, 16384)
- Build label vector y: 0 = Normal, 1 = Pneumonia

Task 4 — PCA (Dimensionality Reduction)

- Apply StandardScaler to normalize feature values
- Apply PCA to reduce from 16,384 features down to ~100 components
- Print how much variance is explained by the 100 components (target: 90%+)
- Plot explained variance curve

Why PCA? X-ray images have 16,384 features. PCA reduces this to ~100 while keeping 90%+ of the information. This makes SVM and KNN train much faster.

Task 5 — Reusable Functions to Write

```
load_dataset(path, size=(128,128))  →  returns X, y
preprocess_images(X)               →  returns X_scaled
apply_pca(X_scaled, n=100)         →  returns X_pca, pca_object
build_features_for_all_datasets()  →  returns all X and y arrays
```

Task 6 — Notebook Documentation (Person A sections)

- Markdown cell explaining each dataset: size, class balance, observations
- Explain why 128x128 was chosen over 64x64 or 256x256
- Explain why PCA is needed and what n_components=100 means

Person B — Splits, Models, Tuning & Deliverables

Person B takes Person A's clean feature matrices and runs all ML experiments. Person B also assembles the final notebook, exports to HTML, and writes the PDF synthesis.

Task 1 — Three Split Strategies

All 3 splits must be implemented and results compared side by side.

- Split 1 — Simple Train/Test Split (80% train, 20% test). Baseline comparison.
- Split 2 — Train/Val/Test Split (70% train, 15% val, 15% test). Use val set for tuning.
- Split 3 — Stratified K-Fold Cross Validation (K=5). Best for limited data.

Split Method	When to Use	Advantage
Simple Train/Test	Quick baseline	Fast, easy to implement
Train/Val/Test	Hyperparameter tuning	Prevents test set contamination
K-Fold CV	Small or imbalanced data	Most robust, averaged results

Task 2 — Models to Train

Train at least 3 models. Compare them all with identical preprocessing.

Model	Library	Key Parameters to Tune
Logistic Regression	sklearn.linear_model	C, penalty, solver
Random Forest	sklearn.ensemble	n_estimators, max_depth, min_samples_split
SVM	sklearn.svm	C, kernel, gamma
KNN	sklearn.neighbors	n_neighbors, metric, weights
CNN (Bonus)	tensorflow.keras	filters, dropout, learning_rate

Task 3 — Hyperparameter Tuning

- Use Dataset 2 (val) specifically for GridSearchCV
- Optimize for ROC-AUC score, not accuracy
- Check bias/variance: if train AUC >> val AUC → overfitting
- Check bias/variance: if both scores are low → underfitting
- Save best parameters found

Task 4 — Evaluation Metrics

Never use accuracy alone — especially with imbalanced medical data. Compute all of these:

Metric	Why It Matters
Accuracy	Basic baseline — can be misleading if classes are imbalanced
Precision	How many predicted pneumonia cases are actually pneumonia
Recall	Most critical — how many real pneumonia cases did we catch
F1 Score	Balance between precision and recall
ROC-AUC	Main metric — measures class separability, threshold-independent
Confusion Matrix	Full breakdown of true/false positives and negatives

What ROC-AUC measures: It plots True Positive Rate vs False Positive Rate at all thresholds. Score of 1.0 = perfect. Score of 0.5 = random guessing. Its advantage over accuracy: not affected by class imbalance.

Task 5 — Plots to Generate

- ROC curves for all models on one chart
- Confusion matrix for each model
- Model comparison bar chart (F1 and AUC side by side)
- Training vs Validation score chart (bias/variance analysis)

Task 6 — Save the Best Model

- Build a sklearn Pipeline with: StandardScaler → PCA → best classifier
- Save with: joblib.dump(pipeline, 'models/best_pipeline.joblib')
- Reload and run a demo prediction on 1-2 new images
- Confirm saved model gives identical results to in-memory model

Task 7 — Final Notebook Structure

Section	Content	Who Writes
1. Introduction	Problem statement, dataset info, goals	Person B
2. Imports & Setup	All libraries, file paths, random seeds	Person B
3. Data Exploration	Counts, distributions, sample images	Person A
4. Preprocessing	Resize, normalize, flatten, PCA	Person A
5. Split Strategy 1	Train/Test baseline results	Person B
6. Split Strategy 2	Train/Val/Test + tuning	Person B
7. Split Strategy 3	K-Fold CV results	Person B

8. Model Comparison	Table + ROC curves for all models	Person B
9. Best Model Analysis	Confusion matrix, misclassified examples	Person B
10. Model Persistence	Save + reload demo	Person B
11. Conclusion	Winner model, why, limitations	Person B

Task 8 — Deliverables to Export

- zoidberg2_pneumonia.ipynb — complete notebook, all cells executed cleanly
- zoidberg2_pneumonia.html — run: jupyter nbconvert --to html notebooks/zoidberg2_pneumonia.ipynb
- summary.pdf — PDF synthesis with goals, methods, results, conclusions
- models/best_pipeline.joblib — saved trained pipeline

Bonus Options (Optional)

Bonus Task	What It Involves	Difficulty
3-Class Prediction	Normal vs Viral Pneumonia vs Bacterial Pneumonia	Medium
CNN Deep Learning	Conv2D layers in TensorFlow/Keras, much better accuracy	Hard
Self-Organizing Map	Visualize feature clusters before training	Medium

Tech Stack Summary

Purpose	Library
Image loading & processing	Pillow, opencv-python
Data manipulation	numpy, pandas
Preprocessing & scaling	sklearn.preprocessing
Dimensionality reduction	sklearn.decomposition.PCA
ML Models	sklearn (LogisticRegression, RandomForest, SVM, KNN)
Hyperparameter tuning	GridSearchCV, RandomizedSearchCV
Evaluation metrics	sklearn.metrics
Visualization	matplotlib, seaborn

Model saving	joblib
Bonus deep learning	tensorflow.keras
Bonus SOM	minisom
Notebook & export	jupyter, nbconvert

Final Delivery Checklist

Task	Person	Status
Organize datasets into dataset1/2/3	Person A	[ ]
EDA — counts, corrupt check, sizes	Person A	[ ]
Class distribution plots	Person A	[ ]
Sample X-ray visualization	Person A	[ ]
Preprocessing pipeline (resize, normalize, flatten)	Person A	[ ]
PCA + variance explained chart	Person A	[ ]
Reusable load_dataset() and preprocess functions	Person A	[ ]
Simple Train/Test split + baseline results	Person B	[ ]
Train/Val/Test split + tuning on dataset2	Person B	[ ]
K-Fold Cross Validation	Person B	[ ]
Train Logistic Regression, Random Forest, SVM, KNN	Person B	[ ]
GridSearchCV tuning — optimize for ROC-AUC	Person B	[ ]
Compute all metrics for each model	Person B	[ ]
ROC curves + confusion matrix plots	Person B	[ ]
Save best model as sklearn Pipeline + joblib	Person B	[ ]
Merge everything into clean final notebook	Person B	[ ]
Export notebook to HTML	Person B	[ ]
Write PDF synthesis report	Person B	[ ]